

Einführung in die logische Programmierung mit Prolog

Grundlagen, Konzepte und erste Modellierungsbeispiele

Antoinette Hellmich

Version 1.0

Zusammenfassung

Dieses Whitepaper führt praxisorientiert in die logische Programmierung mit PROLOG ein. Im Mittelpunkt steht nicht nur die Syntax der Sprache, sondern die Frage, wie Probleme so beschrieben werden, dass das System selbst nach passenden Lösungen suchen kann. Erläutert werden Fakten, Regeln, Variablen, Unifikation, Listen und ein erster Einblick in Backtracking. Ein kleines Rätselbeispiel zeigt abschließend, wie aus einer deklarativen Beschreibung ein ausführbares Suchproblem entsteht.

Inhaltsverzeichnis

1	Motivation: Warum eine andere Art zu programmieren?	3
2	Vom Algorithmus zur Beschreibung	3
2.1	Imperative und deklarative Sichtweise	3
2.2	Programme als Wissensbasis	4
3	Grundelemente der Sprache	4
3.1	Fakten	4
3.2	Regeln	4
3.3	Atome, Zahlen und Variablen	5
3.4	Strukturen	5
3.5	Listen	5
4	Anfragen und Unifikation	5
4.1	Anfragen als Suche nach Belegungen	5
4.2	Unifikation	6
5	Wie Prolog Lösungen ableitet	6
5.1	Klauselauswahl und Zielreduktion	6
5.2	Ein erster Blick auf Backtracking	6
6	Erstes Modellierungsbeispiel: Ein kleines Zahlenrätsel	7
6.1	Problemstellung	7
6.2	Generate-and-Test	7
6.3	Was geschieht während der Auswertung?	8
6.4	Deklarativer Kern des Beispiels	8
7	Typische Denkfehler beim Einstieg	8
7.1	Prolog als imperatives Programm schreiben	8

7.2	Unifikation und Zuweisung verwechseln	8
7.3	Arithmetische Auswertung voraussetzen	8
7.4	Reihenfolge als bedeutungslos betrachten	9
8	Arbeitsweise beim Entwickeln mit SWI-Prolog	9
9	Erkenntnisse aus dem ersten Beispiel	9
10	Ausblick auf die Whitepaper-Reihe	10

1 Motivation: Warum eine andere Art zu programmieren?

Die meisten Entwicklerinnen und Entwickler lernen zunächst imperative oder objektorientierte Sprachen wie Java, C# oder Python. Dort wird ein Lösungsweg üblicherweise als Folge von Anweisungen formuliert: Variablen werden belegt, Schleifen durchlaufen Datenstrukturen und Verzweigungen entscheiden über den weiteren Ablauf.

PROLOG verfolgt einen anderen Ansatz. Statt einen Algorithmus vollständig vorzugeben, beschreibt ein Programm Beziehungen, Fakten und Regeln. Eine Anfrage formuliert anschließend, welche Aussage überprüft oder welche Belegung gesucht werden soll. Die Suche nach passenden Belegungen übernimmt die Inferenzkomponente der Sprache. Diese deklarative Sichtweise ist insbesondere bei Problemen interessant, die sich als Kombination aus Wissen, Bedingungen und möglichen Alternativen formulieren lassen [Bot93; Ert21].

Typische Einsatzfelder sind unter anderem:

- Wissensrepräsentation und regelbasierte Systeme,
- kombinatorische Suchprobleme,
- symbolische Verarbeitung,
- Sprachverarbeitung,
- Constraint-Probleme und logische Rätsel.

Das Ziel dieses Whitepapers ist daher nicht, PROLOG nur als weitere Programmiersprache vorzustellen. Es soll vielmehr nachvollziehbar machen, *wie* sich die Denkweise beim Programmieren verändert.

2 Vom Algorithmus zur Beschreibung

2.1 Imperative und deklarative Sichtweise

Ein imperatives Programm beantwortet vor allem die Frage:

Welche Schritte müssen in welcher Reihenfolge ausgeführt werden, damit ein Ergebnis entsteht?

Ein deklaratives Programm beantwortet dagegen eher:

Welche Beziehungen und Bedingungen müssen gelten, damit ein Ergebnis als gültig betrachtet wird?

Der Unterschied lässt sich an einem einfachen Beispiel zeigen. Gesucht werden Elemente einer Liste. In einer imperativen Sprache würde typischerweise eine Schleife durch die Liste laufen. In PROLOG kann die Beziehung rekursiv beschrieben werden:

```
1  mitglied(X, [X|_]).  
2  mitglied(X, [_|Rest]) :-  
3      mitglied(X, Rest).
```

Listing 1: Rekursive Beschreibung der Mitgliedschaft in einer Liste

Die beiden Klauseln sagen:

1. Ein Element ist Mitglied einer Liste, wenn es am Anfang der Liste steht.
2. Ein Element ist Mitglied einer Liste, wenn es Mitglied der Restliste ist.

Die Anfrage

```
1  ?- mitglied(X, [rot, gruen, blau]).
```

liefert nacheinander mögliche Belegungen für **X**. Die Beschreibung legt nicht ausdrücklich fest, wie eine Schleife auszuführen ist. Dennoch kann PROLOG die Beziehung operational auswerten.

2.2 Programme als Wissensbasis

Ein PROLOG-Programm kann als Wissensbasis verstanden werden. Sie besteht typischerweise aus:

Fakten Aussagen, die ohne weitere Bedingungen gelten.

Regeln Aussagen, die unter bestimmten Voraussetzungen gelten.

Anfragen Fragen an die Wissensbasis.

Diese Trennung ist für das Verständnis der Sprache zentral. Der Quellcode beschreibt nicht primär einen Kontrollfluss, sondern einen Bestand an Beziehungen, aus dem Lösungen abgeleitet werden.

3 Grundelemente der Sprache

3.1 Fakten

Fakten beschreiben elementare Beziehungen. Ein Fakt endet mit einem Punkt.

```
1  farbe(apfel, rot).
2  farbe(banane, gelb).
3  farbe(gurke, gruen).
```

Listing 2: Einfache Fakten

Das Prädikat `farbe/2` besitzt zwei Argumente: ein Objekt und eine Farbe. Die Schreibweise `farbe/2` bezeichnet den Namen des Prädikats und seine Stelligkeit.

Eine Anfrage kann prüfen, ob ein Fakt gilt:

```
1  ?- farbe(apfel, rot).
2  true.
```

Oder sie kann nach einer passenden Belegung fragen:

```
1  ?- farbe(apfel, Farbe).
2  Farbe = rot.
```

3.2 Regeln

Regeln beschreiben Beziehungen, die gelten, wenn ihre Bedingungen erfüllt sind. Der Operator `:-` kann als “gilt, wenn” gelesen werden.

```
1  obst(X) :-
2      farbe(X, _),
3      essbar(X).
4
5  nessbar(apfel).
6  essbar(banane).
```

Listing 3: Regel mit zwei Bedingungen

Das Komma entspricht einer logischen Konjunktion: Beide Bedingungen müssen erfüllt sein.

3.3 Atome, Zahlen und Variablen

In PROLOG beginnen Atome üblicherweise mit einem Kleinbuchstaben. Variablen beginnen mit einem Großbuchstaben oder einem Unterstrich.

Element	Beispiel	Bedeutung
Atom	<code>rot</code>	symbolischer Wert
Zahl	<code>42</code>	numerischer Wert
Variable	<code>Farbe</code>	noch unbekannte Belegung
Anonyme Variable	<code>_</code>	Wert ist an dieser Stelle irrelevant
Struktur	<code>haus(rot, deutsch)</code>	zusammengesetzter Term

Variablen werden nicht wie in imperativen Sprachen durch wiederholte Zuweisungen verändert. Sie werden während einer Ableitung gebunden. Eine einmal hergestellte Bindung gilt innerhalb des jeweiligen Lösungszweigs.

3.4 Strukturen

Strukturen fassen mehrere Werte in einem Term zusammen. Sie eignen sich deshalb zur Modellierung fachlicher Objekte.

```
1 haus(rot, brite, milch, pall_mall, vogel).
```

Listing 4: Strukturierte Darstellung eines Hauses

Der Term kann als Haus mit fünf Eigenschaften interpretiert werden. Solche Strukturen bilden später die Grundlage für komplexere Modelle wie das Einstein-Rätsel.

3.5 Listen

Listen werden in eckigen Klammern geschrieben:

```
1 [rot, gruen, blau]
```

Die Schreibweise `[Kopf|Rest]` zerlegt eine nichtleere Liste in ihr erstes Element und die verbleibende Restliste.

```
1 erstes_element([Kopf|_], Kopf).  
2 restliste([_|Rest], Rest).
```

Listing 5: Zerlegung einer Liste

Listen sind in PROLOG besonders wichtig, weil viele Such- und Modellierungsprobleme als Verarbeitung von Folgen möglicher Werte beschrieben werden können.

4 Anfragen und Unifikation

4.1 Anfragen als Suche nach Belegungen

Eine Anfrage kann eine Aussage prüfen oder Belegungen für Variablen bestimmen.

```
1 ?- farbe(X, rot).  
2 X = apfel.
```

Die Anfrage enthält die Variable `X`. PROLOG sucht nach einer Belegung, durch die der Anfrage-Term mit einem Fakt oder dem Kopf einer Regel übereinstimmt.

4.2 Unifikation

Diesen Abgleich bezeichnet man als Unifikation. Zwei Terme sind unifizierbar, wenn Variablen so gebunden werden können, dass beide Terme strukturell übereinstimmen [Bot93].

Beispiele:

Term 1	Term 2	Ergebnis
farbe(apfel, X)	farbe(apfel, rot)	X = rot
haus(X, brite)	haus(rot, brite)	X = rot
haus(rot, X)	haus(blau, deutsch)	nicht unifizierbar

Unifikation ist mehr als ein einfacher Vergleich. Sie verbindet Prüfung und Bindung: Ein Term wird nicht nur darauf untersucht, ob er passt; gleichzeitig werden unbekannte Bestandteile konkretisiert.

5 Wie Prolog Lösungen ableitet

5.1 Klauselauswahl und Zielreduktion

Bei einer Anfrage versucht PROLOG, das aktuelle Ziel mit einem Fakt oder einem Regelkopf zu unifizieren. Wird eine Regel verwendet, ersetzt das System das Ziel durch die Bedingungen der Regel. Dieser Vorgang wird fortgesetzt, bis entweder alle Teilziele erfüllt sind oder eine Bedingung scheitert.

Betrachten wir die folgende Wissensbasis:

```
1 elternteil(anna, ben).
2 elternteil(ben, clara).
3
4 grosselternteil(X, Z) :-
5     elternteil(X, Y),
6     elternteil(Y, Z).
```

Listing 6: Kleine Wissensbasis zu Familienbeziehungen

Die Anfrage

```
1 ?- grosselternteil(anna, clara).
```

wird schrittweise reduziert:

1. Das Ziel wird mit dem Kopf der Regel `grosselternteil/2` unifiziert.
2. Daraus entstehen die Teilziele `elternteil(anna, Y)` und `elternteil(Y, clara)`.
3. Das erste Teilziel bindet `Y` an `ben`.
4. Das zweite Teilziel kann anschließend mit `elternteil(ben, clara)` erfüllt werden.

5.2 Ein erster Blick auf Backtracking

Existieren mehrere mögliche Klauseln oder Belegungen, legt PROLOG Wahlpunkte an. Scheitert die aktuelle Alternative, kehrt das System zu einem solchen Wahlpunkt zurück und versucht die nächste Möglichkeit. Dieser Mechanismus wird Backtracking genannt.

Das eingebaute Prädikat `member/2` demonstriert dieses Verhalten besonders anschaulich [SWIa]:

```

1  ?- member(X, [1,2,3]).
2  X = 1 ;
3  X = 2 ;
4  X = 3 ;
5  false.

```

Die Suche erfolgt standardmäßig tiefenorientiert und von links nach rechts. Deshalb beeinflusst die Reihenfolge von Regeln und Teilzielen nicht nur die Lesbarkeit, sondern häufig auch die Effizienz eines Programms. Eine vertiefte Analyse dieses Verhaltens ist Gegenstand des zweiten Whitepapers der Reihe.

6 Erstes Modellierungsbeispiel: Ein kleines Zahlenrätsel

6.1 Problemstellung

Gesucht werden zwei verschiedene Ziffern A und B, für die gilt:

$$10A + B = 3(A + B). \quad (1)$$

Zusätzlich sollen beide Ziffern aus dem Bereich 0 bis 9 stammen und verschieden sein. Eine führende Null für A ist nicht zulässig.

Dieses Beispiel ist bewusst klein. Es zeigt bereits die wesentlichen Elemente eines deklarativen Suchproblems:

- Es gibt einen Wertebereich.
- Es gibt Nebenbedingungen.
- Es gibt eine arithmetische Bedingung.
- Die gesuchte Lösung ist eine Variablenbelegung.

6.2 Generate-and-Test

Eine einfache Strategie besteht darin, zunächst mögliche Werte zu erzeugen und diese anschließend zu prüfen.

```

1  ziffer(0). ziffer(1). ziffer(2). ziffer(3). ziffer(4).
2  ziffer(5). ziffer(6). ziffer(7). ziffer(8). ziffer(9).
3
4  loesung(A, B) :-
5      ziffer(A),
6      ziffer(B),
7      A \= 0,
8      A \= B,
9      10 * A + B =:= 3 * (A + B).

```

Listing 7: Generate-and-Test für ein kleines Zahlenrätsel

Die Anfrage

```

1  ?- loesung(A, B).

```

liefert gültige Belegungen. Die Wissensbasis beschreibt dabei nicht, wie Schleifen zu programmieren sind. Sie formuliert lediglich Kandidaten und Bedingungen.

6.3 Was geschieht während der Auswertung?

PROLOG wählt zunächst einen Wert für A und danach einen Wert für B. Anschließend werden die Nebenbedingungen geprüft. Scheitert eine Bedingung, wird zur letzten offenen Alternative zurückgekehrt.

Damit entsteht folgende allgemeine Struktur:

1. Kandidaten erzeugen,
2. Nebenbedingungen anwenden,
3. arithmetische Bedingung prüfen,
4. bei Fehlschlag alternative Belegung suchen.

Diese Strategie ist leicht verständlich, kann bei großen Wertebereichen aber einen sehr großen Suchraum erzeugen. Spätere Whitepaper zeigen, wie sich die Reihenfolge von Bedingungen und die schrittweise Bindung von Variablen auf den Suchaufwand auswirken.

6.4 Deklarativer Kern des Beispiels

Das Programm lässt sich auf eine zentrale Aussage reduzieren:

Eine Lösung ist eine Belegung von A und B, die aus dem zulässigen Wertebereich stammt und alle definierten Bedingungen erfüllt.

Genau dieses Muster findet sich in vielen Anwendungen wieder. Die fachliche Qualität eines PROLOG-Programms hängt deshalb stark davon ab, wie präzise Wertebereiche, Beziehungen und Randbedingungen modelliert werden.

7 Typische Denkfehler beim Einstieg

7.1 Prolog als imperatives Programm schreiben

Ein häufiger Fehler besteht darin, jeden Schritt wie in einer imperativen Sprache kontrollieren zu wollen. Dadurch entstehen unnötig komplizierte Prädikate. Sinnvoller ist zunächst die Frage:

Welche Beziehung soll gelten?

Erst danach sollte untersucht werden, wie PROLOG diese Beziehung operational auswertet.

7.2 Unifikation und Zuweisung verwechseln

Die Schreibweise `X = Wert` ist keine klassische Zuweisung. Sie fordert die Unifikation beider Terme. Eine gebundene Variable kann innerhalb desselben Lösungszweigs nicht anschließend beliebig mit einem anderen Wert überschrieben werden.

7.3 Arithmetische Auswertung voraussetzen

Der Operator `=` führt keine arithmetische Berechnung aus:

```
1 ?- X = 2 + 3.  
2 X = 2+3.
```

Für die Berechnung wird `is/2` verwendet:

```
1 ?- X is 2 + 3.  
2 X = 5.
```

Für numerische Vergleiche existieren unter anderem `==`, `=^`, `<`, `=<`, `>` und `>=`.

7.4 Reihenfolge als bedeutungslos betrachten

Logisch können zwei Bedingungen dieselbe Aussage beschreiben. Operational kann ihre Reihenfolge dennoch große Auswirkungen haben. Eine früh scheiternde Bedingung kann den Suchraum deutlich reduzieren. Diese Verbindung zwischen deklarativer Bedeutung und operationalem Verhalten gehört zu den wichtigsten Lernschritten in PROLOG.

8 Arbeitsweise beim Entwickeln mit SWI-Prolog

Für die Beispiele eignet sich SWI-PROLOG, eine frei verfügbare und gut dokumentierte Implementierung der Sprache [SWIb].

Ein einfacher Arbeitsablauf besteht aus folgenden Schritten:

1. Quelltext in einer Datei mit der Endung `.pl` speichern.
2. Datei in SWI-PROLOG laden.
3. Anfragen interaktiv ausführen.
4. Mit Semikolon weitere Lösungen anfordern.
5. Fehlverhalten durch kleine Testanfragen isolieren.

Beispiel zum Laden einer Datei:

```
1 ?- [beispiel].  
2 true.
```

Nach Änderungen kann die Datei mit `make/0` neu geladen werden:

```
1 ?- make.  
2 true.
```

Für den Einstieg empfiehlt es sich, Prädikate zunächst mit sehr kleinen Datenmengen zu testen. Dadurch bleibt nachvollziehbar, welche Variablen wann gebunden werden und an welcher Stelle Backtracking entsteht.

9 Erkenntnisse aus dem ersten Beispiel

Das Zahlenrätsel zeigt bereits mehrere grundlegende Eigenschaften der logischen Programmierung:

1. **Probleme werden durch Beziehungen beschrieben.** Die Lösung ergibt sich aus der Kombination von Wertebereichen und Bedingungen.
2. **Variablen stehen für gesuchte Belegungen.** Sie werden durch Unifikation schrittweise konkretisiert.
3. **Die Inferenzkomponente übernimmt die Suche.** Alternativen werden automatisch untersucht.
4. **Backtracking ist Teil des Ausführungsmodells.** Ein Fehlschlag führt zur nächsten möglichen Belegung.
5. **Modellierungsentscheidungen beeinflussen die Effizienz.** Reihenfolge und Stärke von Bedingungen bestimmen, wie viel des Suchraums untersucht wird.

Damit wird auch die zentrale Stärke von PROLOG sichtbar: Ein Programm kann sehr kompakt sein, wenn sich das Problem sinnvoll als Menge logischer Beziehungen formulieren lässt.

10 Ausblick auf die Whitepaper-Reihe

Dieses Whitepaper hat die Grundelemente der Sprache vorgestellt. Die folgenden Teile vertiefen die operationalen und modellierungstechnischen Aspekte:

- Whitepaper 2** Backtracking verstehen und analysieren: Wahlpunkte, Tiefensuche, `trace/0`, `time/1` und Zielreihenfolge.
- Whitepaper 3** Kryptarithmen mit PROLOG modellieren: Permutationen, Generate-and-Test, schriftliche Addition und Suchraumreduktion.
- Whitepaper 4** Constraint-Modellierung mit dem Einstein-Rätsel: Datenstrukturen, Eigenschaftsregeln, Positionsbeziehungen und Optimierung der Hinweisreihenfolge.

Hinweis zum Quellcode

Alle Beispiele dieses Whitepapers sind als ausführbare `.pl`-Dateien im zugehörigen GitHub-Repository vorgesehen. Für reproduzierbare Ergebnisse sollten die verwendete SWI-PROLOG-Version und gegebenenfalls relevante Systemeinstellungen dokumentiert werden.

Literatur

- [Bot93] Ralf Bothner. *Programmieren in PROLOG*. Wiesbaden: Vieweg+Teubner, 1993. ISBN: 978-3-322-87223-4.
- [Ert21] Wolfgang Ertel. *Grundkurs Künstliche Intelligenz*. 5. Aufl. Wiesbaden: Springer Vieweg, 2021. DOI: [10.1007/978-3-658-32075-1](https://doi.org/10.1007/978-3-658-32075-1).
- [SWIa] SWI-Prolog. *member/2 – List membership*. URL: https://www.swi-prolog.org/pldoc/doc_for?object=member/2 (besucht am 10.07.2026).
- [SWIb] SWI-Prolog. *SWI-Prolog – The comprehensive Prolog implementation*. URL: <https://www.swi-prolog.org/> (besucht am 10.07.2026).